Clase D03 · Patrones GoF

Bloque 2 · Diseño · Clase 3 de 9

Modelación y Diseño de Sistemas · 2026 · Bloque Diseño

Prof. Ing. Fernando Bono · ESCMB UNC · Pregrado AUSI

Agenda

- Recap D02: GRASP nos dijo "dónde", GoF nos dice "cómo"
- ¿Qué son los patrones GoF?
- Catálogo completo: los 23 patrones organizados por familia
- Los 7 patrones que vemos en detalle hoy (con caso ChangoMás):
- Creacionales: Singleton, Factory Method
- Estructurales: Adapter, Decorator
- Comportamiento: Strategy, Observer, State
- Los 16 restantes como mini-fichas de referencia
- Mapa integrador en ChangoMás
- Anti-patterns: cuándo NO meter un patrón
- Conexión GRASP ↔ GoF
- Errores comunes
- Actividad grupal: cada grupo profundiza 1-2 patrones (entregable + presentación 15 min)
- Cierre + qué viene (D04 Arquitectura)

1 · Recap D02 + por qué GoF

En D02 vimos los GRASP: 9 patrones para decidir quién tiene cada responsabilidad.

GRASP responde: ¿dónde pongo este método?

GoF responde: ¿cómo implemento esta solución de forma reusable?

Qué GoF NO es

- No son una biblioteca para copiar y pegar.
- No se "tienen que usar todos" en cada sistema.
- No reemplazan a GRASP, lo complementan.

Aplicar GoF sin entender GRASP = ingeniería de patrones por moda.

> *GRASP es el razonamiento. GoF es la receta probada para problemas comunes.*

2 · ¿Qué son los patrones GoF?

- Propuestos por el Gang of Four (Gamma, Helm, Johnson, Vlissides) en 1994.
- 23 patrones organizados en 3 familias.
- Cada uno resuelve un problema recurrente de diseño OO.
- Estructura típica:
- Intención: qué problema resuelve.
- Motivación: cuándo se necesita.
- Estructura: diagrama de clases.
- Consecuencias: pros y contras.

Clasificación

Familia	Foco	Patrones que vemos hoy
Creacionales	Cómo se crean los objetos	Singleton · Factory Method
Estructurales	Cómo se componen las clases	Adapter · Decorator
Comportamiento	Cómo colaboran y se comunican	Strategy · Observer · State

Los 23 patrones originales siguen vigentes 30 años después. La industria los usa todos los días, muchas veces sin darse cuenta.

3 · Catálogo completo · los 23 patrones GoF

Creacionales (5)

Estructurales (7)

Comportamiento (11)

Patrón	Problema que resuelve
Singleton	Garantizar UNA sola instancia
Factory Method	Encapsular la creación de objetos
Abstract Factory	Crear familias de objetos relacionados
Builder	Construir objetos complejos paso a paso
Prototype	Crear nuevos objetos clonando un prototipo

4 · Los 7 que vemos en detalle hoy

#	Patrón	Familia	Por qué este
1	Singleton	Creacional	El más usado y abusado
2	Factory Method	Creacional	Base de todos los creacionales
3	Adapter	Estructural	Aparece en cualquier integración legacy
4	Decorator	Estructural	Alternativa elegante a herencia múltiple
5	Strategy	Comportamiento	El más usado del bloque comportamiento
6	Observer	Comportamiento	Base del eventing y MVC clásico
7	State	Comportamiento	Puente directo con el Diagrama de Estados del análisis

Para cada uno vamos a ver: definición · cuándo usarlo · ejemplo ChangoMás · trampa típica.

5 · GoF 1 · Singleton (Creacional)

Cuándo usarlo

- Configuración global de la aplicación.
- Acceso compartido a un recurso costoso (pool de conexiones, caché).
- Cuando tener más de una instancia genera inconsistencias reales del dominio.

Ejemplo ChangoMás · configuración del POS

Cualquier clase pide ConfiguracionPOS.get() y recibe siempre la misma.

Trampa típica

```
class ConfiguracionPOS:
    _instancia = None
    @staticmethod
    def get():
        if ConfiguracionPOS._instancia is None:
            ConfiguracionPOS._instancia = ConfiguracionPOS()
        return ConfiguracionPOS._instancia

    cuit_empresa: str
    tasa_iva_default: Decimal
    timeout_pasarela: int
```

Garantizar que una clase tenga UNA sola instancia y proveer un punto de acceso global a ella.

6 · GoF 2 · Factory Method (Creacional)

Cuándo usarlo

- La decisión de qué tipo crear depende de un parámetro o configuración.
- Querés evitar new ClaseConcreta() desperdigado por el código.
- Vas a agregar nuevos tipos sin tocar a los clientes.

Ejemplo ChangoMás · creación del medio de pago

Conexión con GRASP

```
class MedioPagoFactory:
    @staticmethod
    def crear(tipo: str) -> MedioPago:
        if tipo == "EFECTIVO":  return PagoEfectivo()
        if tipo == "TARJETA":   return PagoTarjeta(...)
        if tipo == "QR":        return PagoQR(...)
        raise TipoDePagoDesconocido(tipo)

class GestorVenta:
    cerrarVenta(tipoPago: str):
        formaPago = MedioPagoFactory.crear(tipoPago)
        formaPago.cobrar(self.venta.total)
```

Definir una interfaz para crear un objeto, pero dejar que las subclases decidan qué clase concreta instanciar.

7 · GoF 3 · Adapter (Estructural)

Cuándo usarlo

- Tenés una librería externa con una API que no encaja en tu modelo.
- Estás migrando un sistema legacy y querés que conviva con el nuevo.
- Necesitás reemplazar una integración sin tocar el código que la usa.

Ejemplo ChangoMás · impresora fiscal vieja

El POS espera un Impresora con una API limpia:

Pero la impresora vieja LegacyTicketPrinter recibe arrays:

Adapter envuelve la legacy:

GestorVenta habla con Impresora. No sabe que por debajo hay una legacy.

Trampa típica

Usar Adapter para "parchear" una mala interfaz interna. Si la interfaz es tuya, corregila.

```
interface Impresora:
    imprimir(ticket: Ticket): void
```

Convertir la interfaz de una clase en otra interfaz que el cliente espera. Permite que clases incompatibles colaboren.

8 · GoF 4 · Decorator (Estructural)

Cuándo usarlo

- Necesitás combinar comportamientos opcionales.
- Hacer herencia explotaría en N variantes (ItemConDescuentoY2x1YCuponYFidelizacion).

```
interface Cobrable:
    precio(): Money

class Item implements Cobrable:
    precio(): Money

class DescuentoTarjeta implements Cobrable:
    base: Cobrable
    porcentaje: int

    precio(): Money:
        return self.base.precio() * (1 - self.porcentaje / 100)

class Promocion2x1 implements Cobrable:
    base: Cobrable
    precio(): Money:
        return self.base.precio() / 2   # cada 2 unidades, una gratis
```

Agregar responsabilidades a un objeto dinámicamente, sin alterar su clase original.

9 · GoF 5 · Strategy (Comportamiento)

Cuándo usarlo

- Tenés varias formas de hacer lo mismo (calcular, validar, ordenar) y la elección depende del contexto o configuración.
- Querés intercambiar el algoritmo en runtime.
- Tenés if/switch enorme por tipo → Strategy lo elimina.

Ejemplo ChangoMás · cálculo de IVA según tipo de producto

Item recibe su strategy según el producto:

Conexión con GRASP

Strategy = Polimorfismo del GRASP elevado a patrón concreto. Casi siempre se combina con Factory Method para elegir la strategy.

```
interface CalculadoraIVA:
    calcular(precio: Money): Money

class IVA21        implements CalculadoraIVA   # general
class IVA105       implements CalculadoraIVA   # carne, pan, frutas
class IVAExento    implements CalculadoraIVA   # libros, leche
class IVADiferido  implements CalculadoraIVA   # productos especiales
```

Definir una familia de algoritmos, encapsular cada uno, y hacerlos intercambiables. Strategy permite que el algoritmo varíe independientemente de los clientes que lo usan.

10 · GoF 6 · Observer (Comportamiento)

Cuándo usarlo

- Varios objetos necesitan reaccionar al mismo evento.
- El "publicador" no debe saber quiénes son los suscriptores.
- Querés desacoplar la lógica que dispara del código que reacciona.

```
class Venta:
    observadores: List<ObservadorVenta>

    cerrar():
        self.estado = "CERRADA"
        for obs in self.observadores:
            obs.alCerrar(self)

interface ObservadorVenta:
    alCerrar(venta: Venta): void

class EmisorComprobante     implements ObservadorVenta
class ActualizadorStock     implements ObservadorVenta
class RegistroFidelizacion  implements ObservadorVenta
class Auditor               implements ObservadorVenta
```

Definir una dependencia 1-a-N entre objetos, de modo que cuando uno cambia de estado, todos sus dependientes son notificados y actualizados automáticamente.

11 · GoF 7 · State (Comportamiento)

Cuándo usarlo

- Una entidad tiene un ciclo de vida con estados bien definidos (¡los del análisis!).
- En cada estado, las operaciones permitidas cambian.
- Querés evitar el if/switch por self.estado en cada método.

Ejemplo ChangoMás · ciclo de vida de Venta

Estados (del Diagrama de Estados del análisis): Abierta → Cerrada → Facturada → Cancelada.

MAL — if/switch en cada método:

BIEN — patrón State:

Conexión con el análisis

El Diagrama de Estados de B1 se traduce directamente a State pattern. Es uno de los puentes más claros entre análisis y diseño.

```
class Venta:
    cancelar():
        if self.estado == "Abierta":   ...
        elif self.estado == "Cerrada": ...
        elif self.estado == "Facturada":
            raise NoSePuedeCancelarFacturada()
```

Permitir que un objeto cambie su comportamiento cuando cambia su estado interno. El objeto parece cambiar de clase.

12 · Mini-fichas · Creacionales restantes

Patrón	Intención	Ejemplo ChangoMás	Trampa típica
Abstract Factory	Crear familias de objetos relacionados sin especificar clases concretas	Un set de drivers según el tipo de caja (SetDispositivosCajaTradicional vs SetDispositivosAutoservicio) provee impresora, lector y display compatibles entre sí	Sobrediseño cuando hay solo una familia real
Builder	Construir un objeto complejo paso a paso separando construcción y representación	ComprobanteFiscalBuilder arma el comprobante AFIP con cabecera, ítems, totales, leyendas opcionales	Para objetos simples de 2-3 campos sobra completamente

Cuándo combinarlos: Abstract Factory internamente puede usar Builder para cada producto complejo.

13 · Mini-fichas · Estructurales restantes

Patrón	Intención	Ejemplo ChangoMás	Trampa típica
Bridge	Desacoplar abstracción de implementación para que evolucionen independiente	MedioPago (abstracción) ↔ PasarelaTarjeta (implementación). Sumar PagoCrédito × 3 pasarelas = 3 combinaciones, no 9 clases	Sobreingeniería si hay solo una implementación
Composite	Tratar objetos individuales y compuestos uniformemente	Un Pedido puede tener Item individuales y Combo (que contiene items). Ambos implementan Vendible.calcularPrecio()	Cuando la jerarquía real no es uniforme se fuerza un diseño raro
Facade	Proveer interfaz simple a un subsistema complejo	FacadeAFIP esconde 12 webservices distintos detrás de emitir(venta), consultarCAE(), anular(cae)	Facade gigante que termina siendo un God Object
Flyweight	Compartir objetos comunes para ahorrar memoria	El catálogo tiene 80.000 productos; los datos compartidos (nombre, IVA, foto) se cachean una sola vez en memoria; cada Item solo guarda código + cantidad	Premature optimization si la memoria no es un problema real
Proxy	Sustituto que controla el acceso a otro objeto (lazy load, cache, auth, logging)	ProxyStockRemoto cachea las consultas al sistema central; si el central está caído, devuelve el último valor conocido	Proxy que termina haciendo más que controlar (= sin cohesión)

14 · Mini-fichas · Comportamiento restantes (parte 1)

Patrón	Intención	Ejemplo ChangoMás	Trampa típica
Chain of Responsibility	Pasar una request por una cadena de handlers hasta que uno la atienda	Pipeline de validación de venta: ValidadorStock → ValidadorPromo → ValidadorPago → ValidadorFiscal. Cualquiera puede rechazar	Cadenas largas y opacas donde nadie sabe qué orden importa
Command	Encapsular una operación como objeto (parámetros, undo, queue, logging)	Cada acción del cajero (EscanearProductoCommand, AplicarPromoCommand, AnularItemCommand) es un objeto que se puede deshacer y registrar	Sumar Command para operaciones triviales que no necesitan undo ni queue
Iterator	Recorrer una colección sin exponer su estructura interna	Venta.iterator() devuelve un iterador de Item sin exponer si por dentro es lista, set o tree	En lenguajes con for-each nativo, implementarlo a mano es ruido
Mediator	Centralizar la comunicación entre varios objetos para reducir acoplamiento N-a-N	MediadorCaja coordina la pantalla del cliente, el display del cajero, el cajón de dinero y la impresora — los componentes solo hablan con el mediador	Mediator que se vuelve un God Object con toda la lógica

15 · Mini-fichas · Comportamiento restantes (parte 2)

Patrón	Intención	Ejemplo ChangoMás	Trampa típica
Memento	Capturar el estado interno de un objeto para restaurarlo después	SnapshotVenta permite "deshacer" hasta 5 pasos de una venta en curso (anular ítem, cambiar cantidad)	Snapshot pesado de objetos grandes = memoria explota
Template Method	Definir el esqueleto de un algoritmo en una clase base, dejando que subclases completen los pasos	ProcesoCierreVentaBase.ejecutar() define el flujo (validar → cobrar → emitir → notificar). Subclases concretas (CierreEfectivo, CierreTarjeta) sobreescriben pasos específicos	Herencia rígida; muchas veces Strategy es más flexible
Visitor	Definir nuevas operaciones sobre una jerarquía de objetos sin modificar las clases	ExportadorPDFVisitor, ValidadorVisitor recorren la jerarquía de items (ItemNormal	Acoplamiento doble: el Visitor conoce a TODAS las clases concretas

Muchos de estos patrones aparecen "sin nombre" en código real porque son tan ubicuos que ya están dentro del lenguaje (Iterator en for-each, Template Method en hooks de tests).

16 · Mapa integrador · GoF en ChangoMás

Componente / Decisión	Patrón GoF	Por qué
ConfiguracionPOS global	Singleton	Una sola configuración para toda la sesión
Crear MedioPago según selección del cajero	Factory Method	Centralizar la creación; agregar nuevos sin tocar callers
Integrar impresora fiscal vieja	Adapter	Convivir con la legacy sin contaminar el dominio
Promociones apilables sobre un ítem	Decorator	Combinar dinámicamente descuentos sin explotar en clases
Cálculo de IVA según tipo de producto	Strategy	Intercambio de algoritmo según el producto
Cierre de venta dispara notificaciones	Observer	Múltiples reactivos sin acoplar al publicador
Ciclo de vida de Venta	State	Operaciones permitidas cambian según el estado

Una sola venta termina usando 6-7 patrones. El cliente nunca los ve, pero el código queda limpio, testeable y extensible.

17 · Anti-patterns · cuándo NO usar un GoF

Los patrones son una herramienta, no un objetivo. Usar uno sin necesidad agrega complejidad y nada más.

Señales de alarma

- "Lo uso porque sí" — no resuelve un problema concreto que tenés HOY.
- Un Singleton para "evitar pasar parámetros" → ocultás dependencias y rompés tests.
- Strategy con UNA sola implementación → es solo una clase con un método.
- Decorator con DOS variantes → es más simple if.
- Observer con UN observador → es una llamada directa.

El test pragmático

Antes de meter un patrón, preguntate:

- ¿Cuántas variantes / decoradores / observers vas a tener realmente?
- ¿Cambian en runtime?
- ¿El equipo entiende el patrón sin documentación?

Regla de oro: meté un patrón cuando aparece la segunda variante, no la primera.

18 · Conexión GRASP ↔ GoF

GoF es la implementación concreta de varios GRASP:

GoF	GRASP que aplica
Singleton	Pure Fabrication + acceso global controlado
Factory Method	Pure Fabrication + Polimorfismo + Bajo Acoplamiento
Adapter	Indirección + Bajo Acoplamiento
Decorator	Polimorfismo + Alta Cohesión
Strategy	Polimorfismo + Indirección
Observer	Indirección + Bajo Acoplamiento
State	Polimorfismo + Alta Cohesión

Si entendés GRASP, GoF se vuelve casi obvio: es la receta nombrada para combinaciones recurrentes.

19 · Errores comunes al aplicar GoF

- Patternitis: meter patrones porque "queda más profesional".
- Singleton-itis: convertir todo lo compartido en Singleton.
- Sobre-factorizar: una Factory para crear una sola clase concreta.
- Strategy fantasma: una sola strategy implementada → es solo una clase.
- Observer cíclico: A escucha a B, B escucha a A → loop infinito.
- State explosivo: 30 estados, cada uno con 15 métodos → revisar si el modelo de dominio es correcto antes de aplicar State.
- Adapter perpetuo: dejar el adapter por años "porque después lo limpiamos" — termina en producción 5 años.

El mejor patrón es el que el junior del equipo entiende en 2 minutos. El segundo mejor es el que está bien documentado.

20 · Actividad grupal · profundizar los 23 patrones

Modalidad

- 12 grupos de 5 alumnos (60 alumnos en total).
- 1 o 2 patrones por grupo (autoorganización entre grupos para cubrir los 23).
- Caso conductor libre por grupo (no obligan ChangoMás — pueden elegir un dominio propio).
- 15 minutos máximo por grupo + 5 min de Q&A.

Estructura obligatoria de cada presentación (15 min)

Entregables

Minuto	Contenido
0-2	Intención del patrón + 1 frase del libro GoF
2-6	Problema que resuelve (ejemplo MAL / sin el patrón)
6-10	Solución (diagrama de clases + código resumido)
10-13	Aplicación al caso conductor que el grupo eligió
13-15	Cuándo NO usarlo + trampas típicas

Cada uno de los 23 patrones se profundiza por un grupo de 4-5 alumnos en una presentación de 15 minutos.

21 · Cierre + qué viene

Lo que vimos hoy

- 7 patrones GoF representativos de las 3 familias (Creacional, Estructural, Comportamiento).
- Cada patrón resuelve un problema concreto y recurrente del diseño OO.
- ChangoMás termina usando 6-7 patrones simultáneos sin contaminar el código del dominio.
- GRASP explica el por qué; GoF da la receta probada.
- Patternitis es peor que no usar patrones: meté uno solo cuando el problema lo justifica.

Clase	Tema
D04	Arquitectura: conceptos + atributos de calidad + ADRs
D05	Estilos clásicos: Capas + Cliente-Servidor
D06	Estilo MVC
D07	Microservicios
D08	Event-Driven
D09	Arquitecturas modernas: Clean + Hexagonal + Onion

Próxima clase (D04): subimos un nivel de zoom — Arquitectura de Software.